UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FCFM

Análisis Numérico

M.A. María Del Carmen Martínez Cejudo

PRODUCTO INTEGRADOR DE APRENDIZAJE

Sebastián Calderón Carrillo 2087472

Edson Armando Amaya Macias 2151976

Juan José Trujillo Garza 1972407

Grupo 031

Introducción

Los centros de datos requieren un suministro eléctrico continuo para mantener en funcionamiento sus servidores y equipos. Para reducir el riesgo de interrupciones, cuentan con baterías de respaldo que almacenan energía cuando existe un excedente y la entregan cuando se necesita. Por ello, conocer el comportamiento de la potencia durante el día permite estimar si las baterías terminaron cargándose o descargándose.

Nuestro proyecto desarrolla una aplicación de consola en Python que calcula el balance neto de energía de las baterías a partir de mediciones de potencia tomadas en distintas horas. El programa construye un polinomio que interpola los datos ingresados mediante un sistema de Vandermonde, resuelto con el método de Montante, y posteriormente integra la función obtenida en el intervalo de 0 a 24 horas mediante la regla compuesta del trapecio.

La potencia positiva representa energía que entra al sistema y carga las baterías; la potencia negativa representa energía que sale y provoca su descarga. El resultado se expresa en kilowatt-hora (kWh) y representa el cambio neto respecto a una carga inicial que no se conoce.

Codificación

El programa está organizado en módulos para separar la captura de datos, los métodos numéricos y la presentación de resultados. El punto de entrada es **src/main.py**, que importa y ejecuta la función **run()** del módulo **menu.py**.

Descripción de los módulos principales

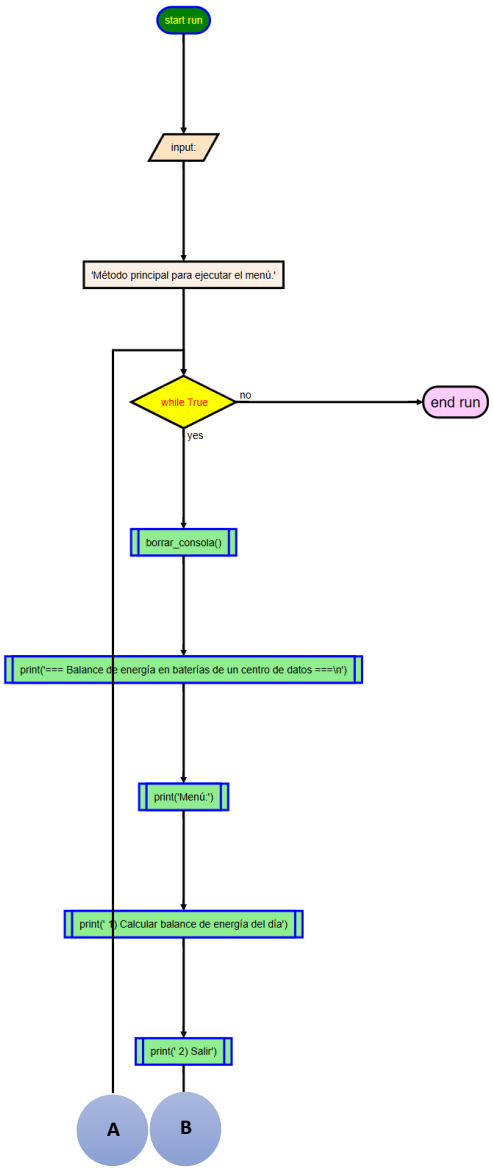
- **main.py**: inicia la aplicación llamando al menú principal.
- **menu.py**: muestra las opciones, solicita las mediciones y el número de trapecios, construye el sistema, ejecuta los cálculos y presenta la interpretación del balance.
- **ajuste.py**: construye la matriz de Vandermonde, obtiene los coeficientes del polinomio y permite evaluarlo.
- **montante.py**: implementa la eliminación de Montante para resolver el sistema de ecuaciones y contiene funciones auxiliares para matrices.
- **integracion.py**: implementa la regla compuesta del trapecio para aproximar la integral.
- **utils.py**: contiene funciones para leer y validar entradas, convertir números a fracciones, dar formato a los resultados y controlar la consola.

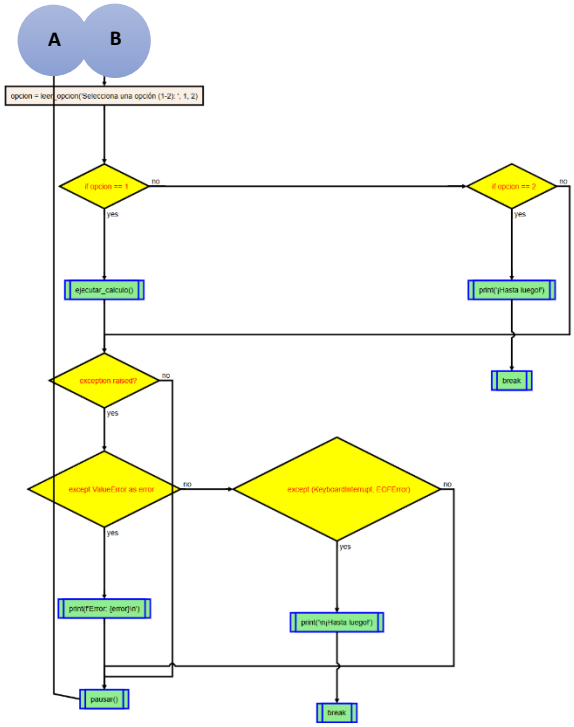
Secuencia del cálculo

Para conservar exactitud durante las operaciones numéricas, el código utiliza la clase Fraction de Python. Así, los coeficientes y la integral se manejan como fracciones exactas y se muestran también en forma decimal aproximada.

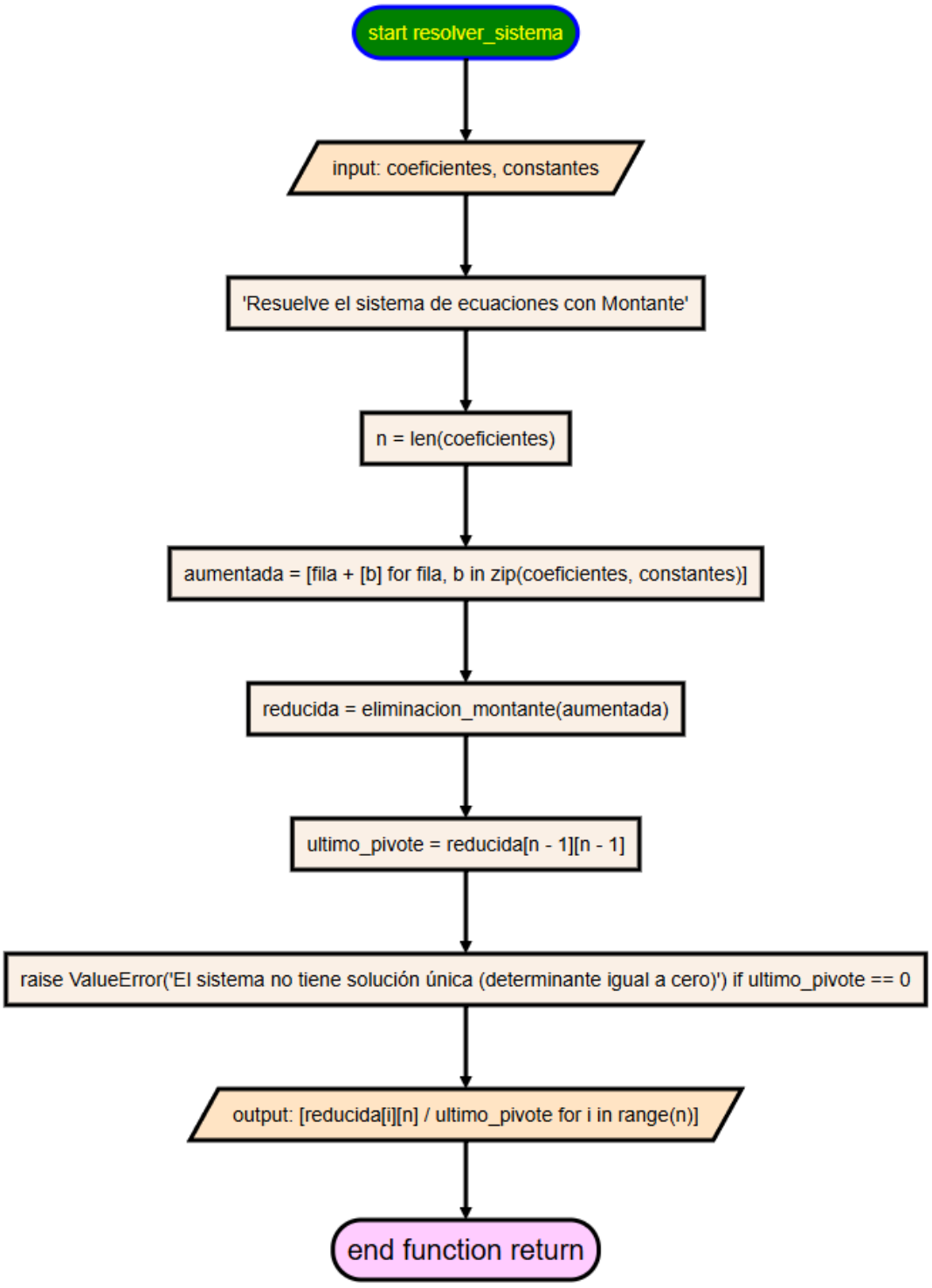
Diagramas de flujo

Correr

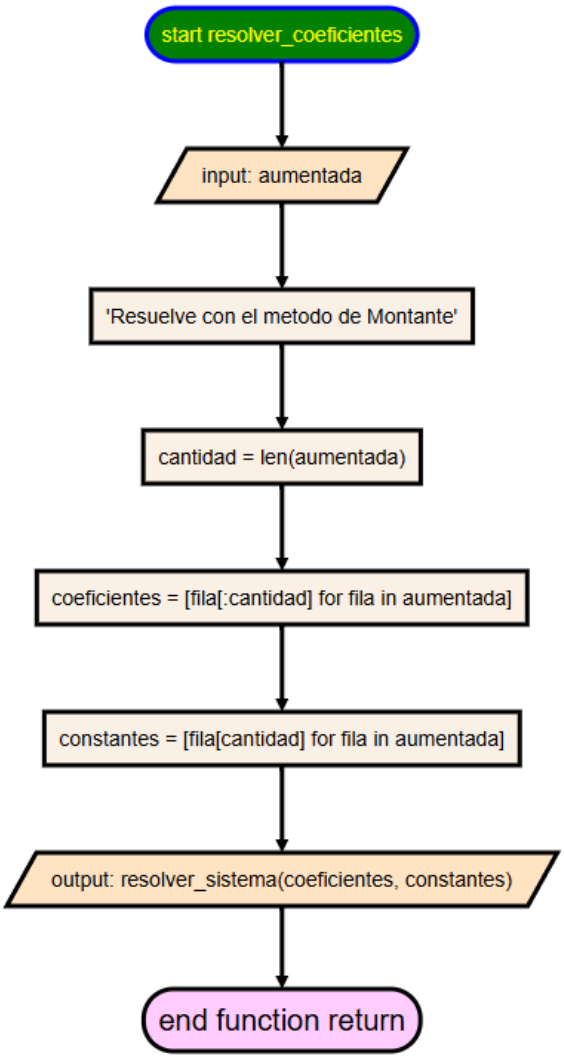




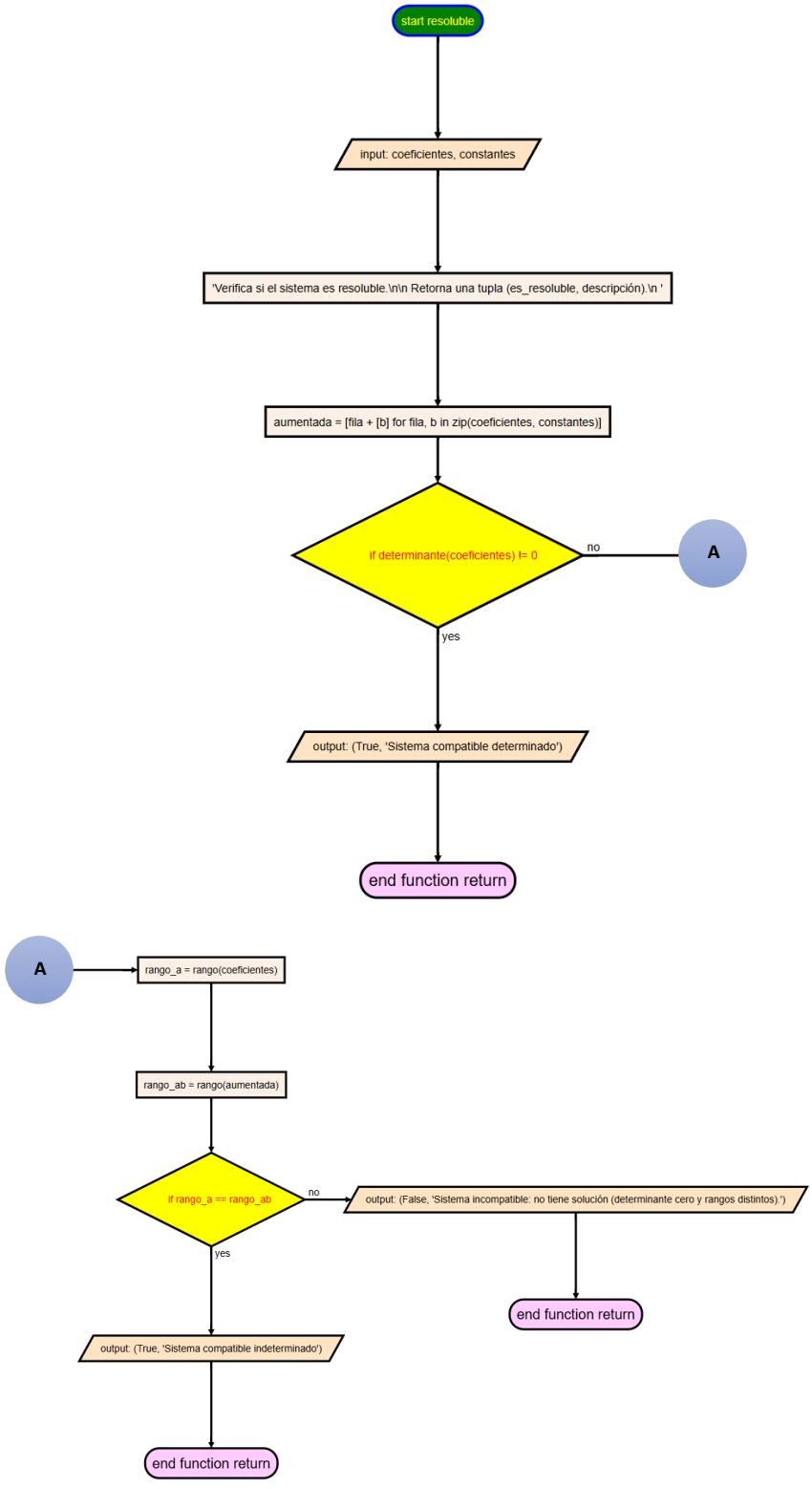
Resolver Sistema



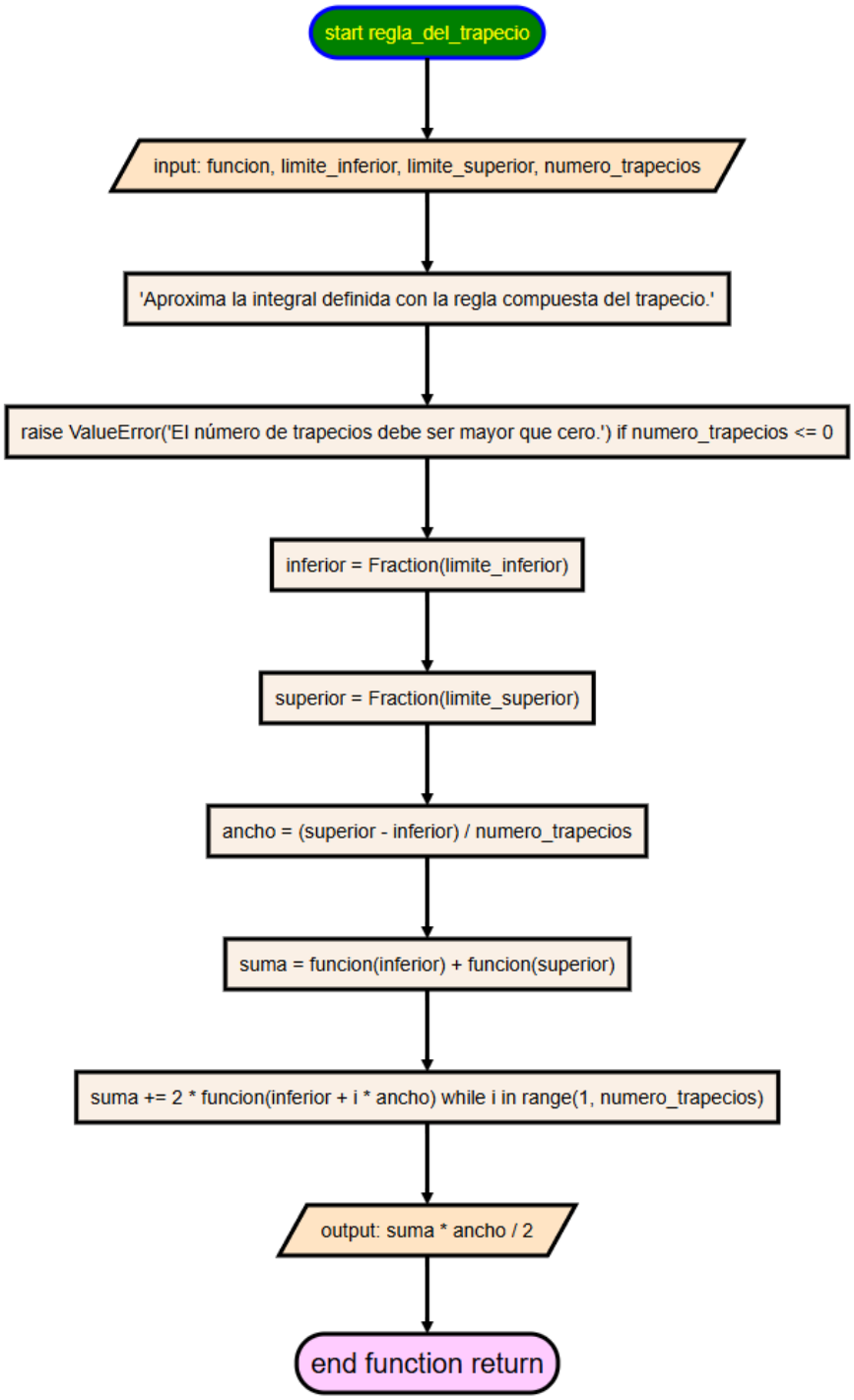
Resolver Coeficientes



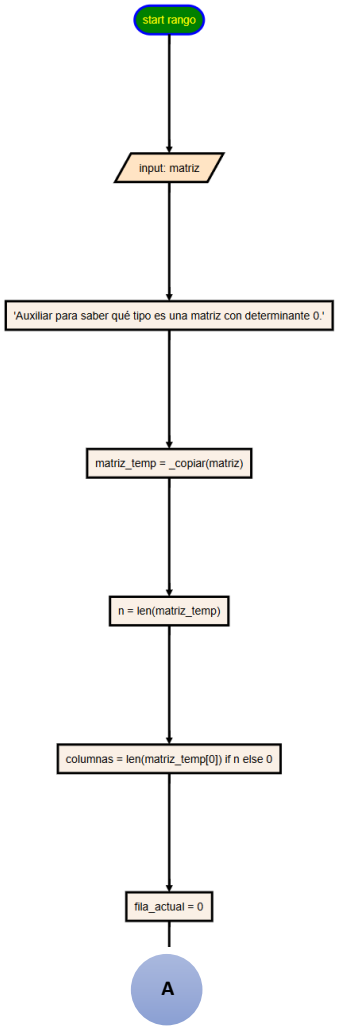
Resoluble

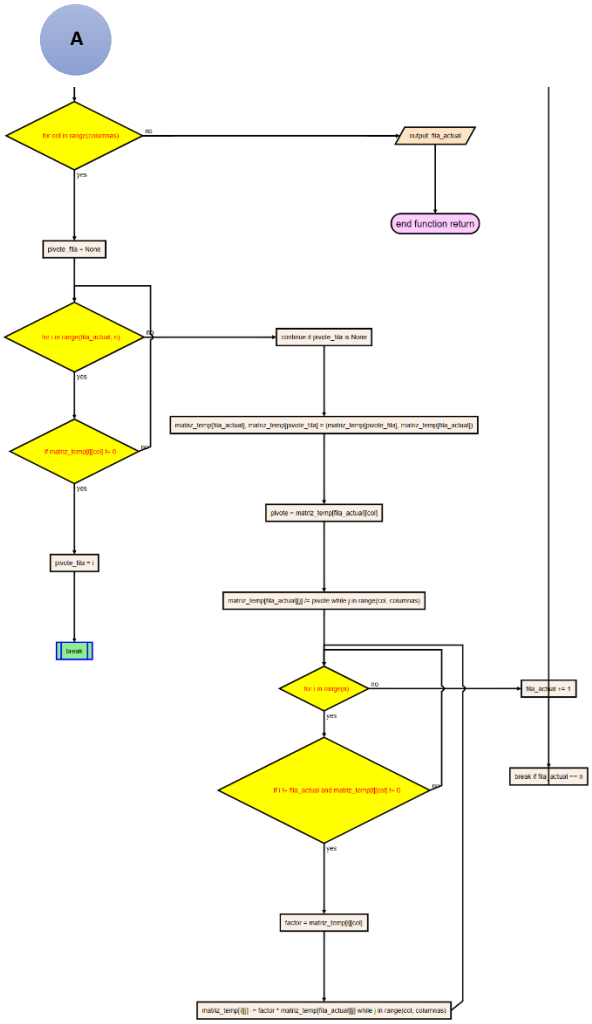

start resoluble
input: coeficientes, constantes
'Verifica si el sistema es resoluble.\n\n Retorna una tupla (es_resoluble, descripción).\n '
aumentada = [fila + [b] for fila, b in zip(coeficientes, constantes)]
if determinante(coeficientes) != 0
no
A
yes
output: (True, 'Sistema compatible determinado')
end function return
A
rango_a = rango(coeficientes)
rango_ab = rango(aumentada)
if rango_a == rango_ab
no
output: (False, 'Sistema incompatible: no tiene solución (determinante cero y rangos distintos).')
end function return
yes
output: (True, 'Sistema compatible indeterminado')
end function return

Regla del Trapecio

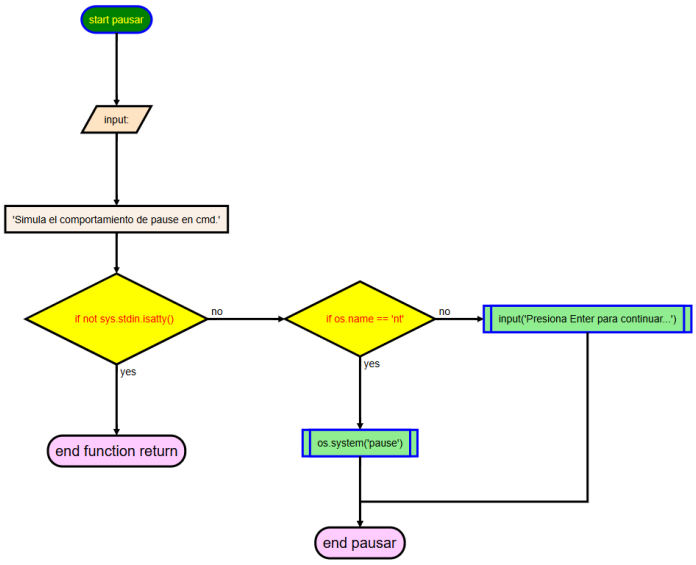


Rango

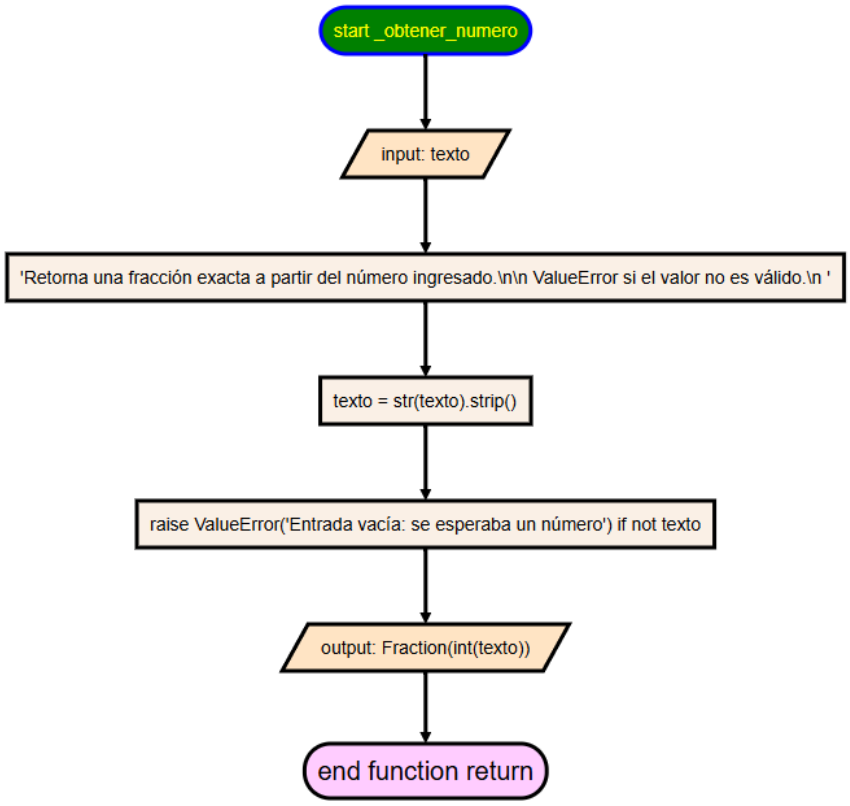

start rango
input: matriz
'Auxiliar para saber qué tipo es una matriz con determinante 0.'
matriz_temp = _copiar(matriz)
n = len(matriz_temp)
columnas = len(matriz_temp[0]) if n else 0
fila_actual = 0
A


A
for col in range(columnas)
no
output: fila_actual
end function return
yes
pivote_fila = None
for i in range(fila_actual, n)
no
continue if pivote_fila is None
yes
if matriz_temp[i][col] != 0
no
matriz_temp[fila_actual], matriz_temp[pivote_fila] = (matriz_temp[pivote_fila], matriz_temp[fila_actual])
yes
pivote = matriz_temp[fila_actual][col]
pivote_fila = i
matriz_temp[fila_actual][j] /= pivote while j in range(col, columnas)
break
for i in range(n)
no
fila_actual += 1
yes
break if fila_actual == n
if i != fila_actual and matriz_temp[i][col] != 0
no
yes
factor = matriz_temp[i][col]
matriz_temp[i][j] -= factor * matriz_temp[fila_actual][j] while j in range(col, columnas)

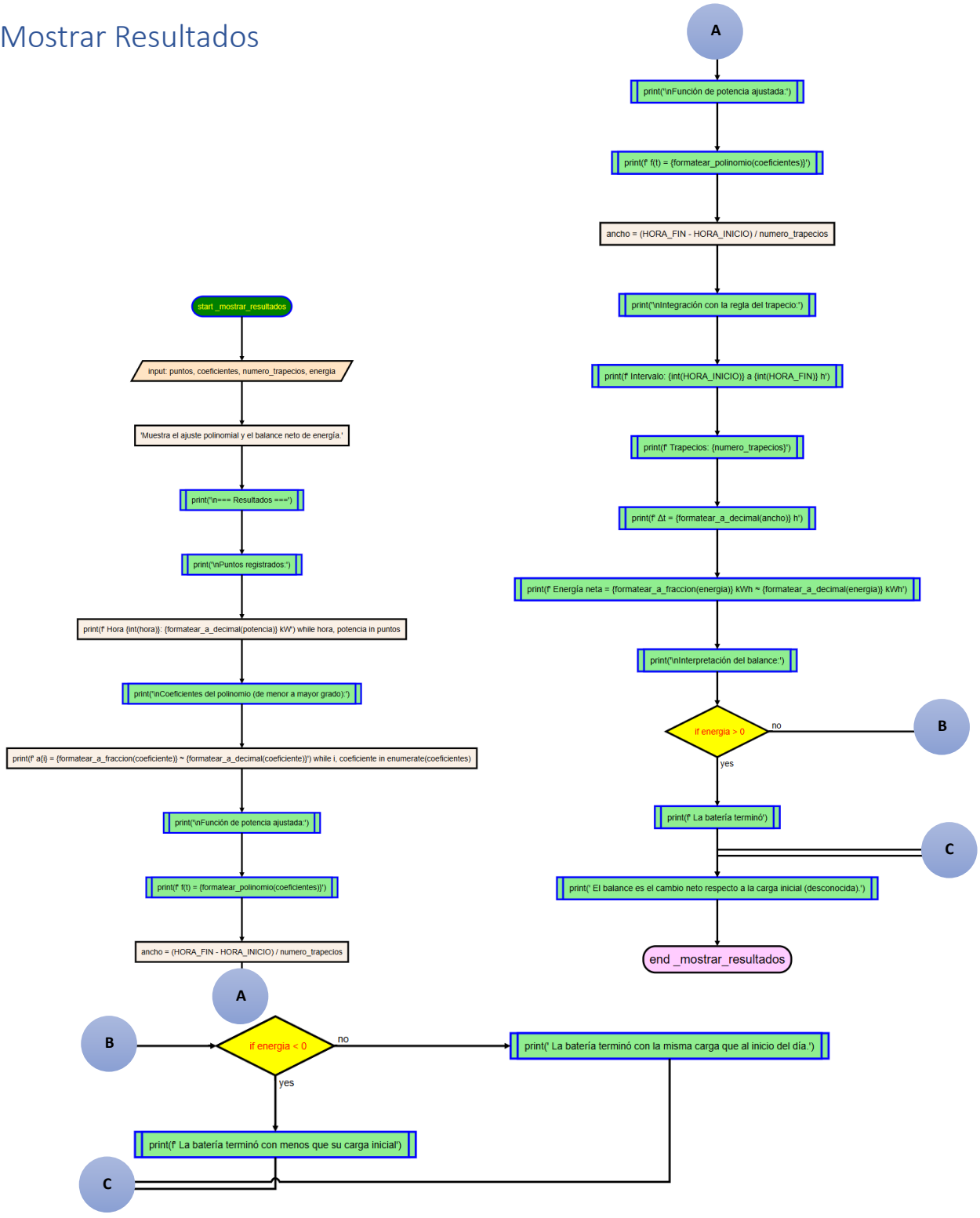
Pausar



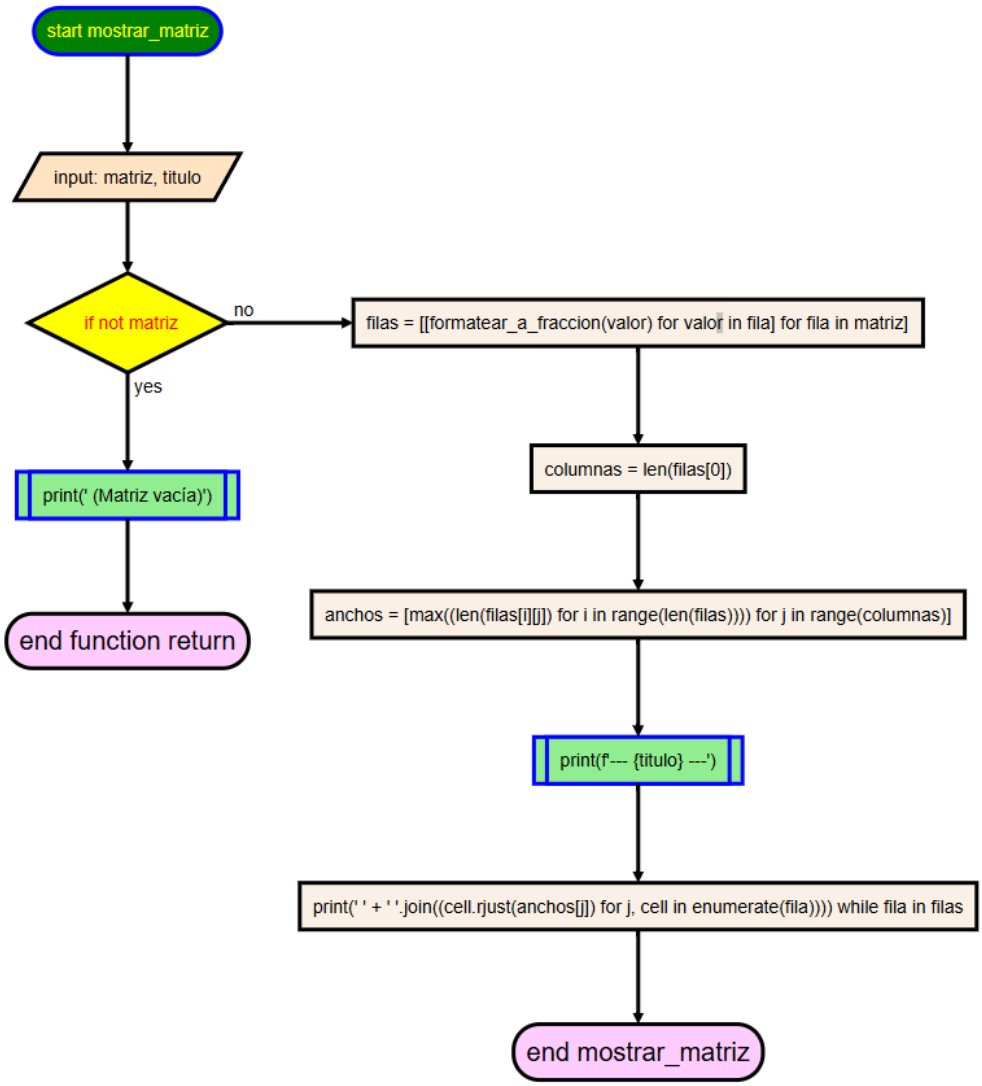
Obtener Numero



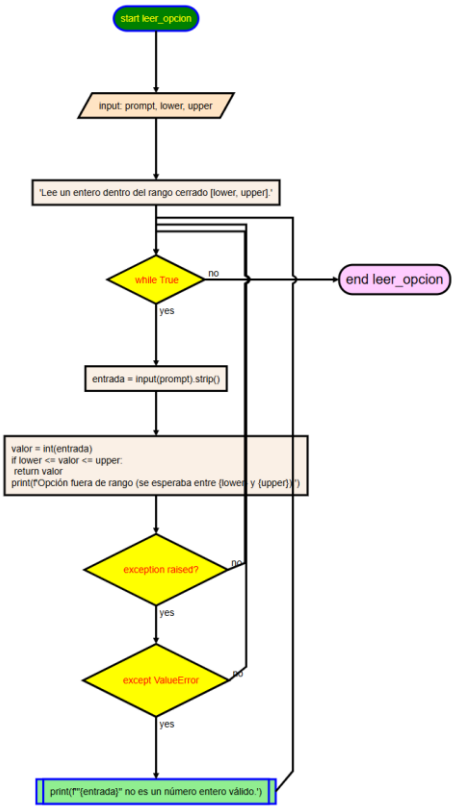
Mostrar Resultados



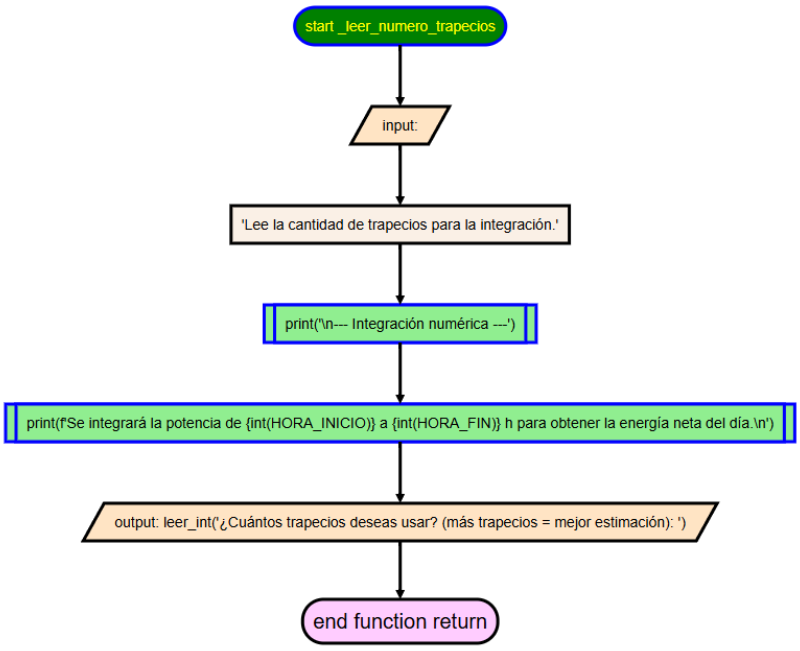
Mostrar Matriz



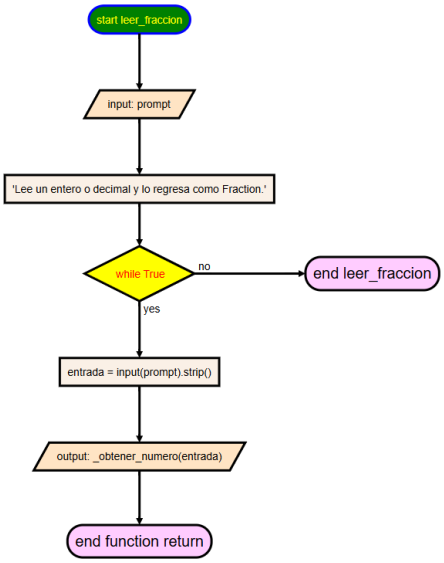
Leer Opción



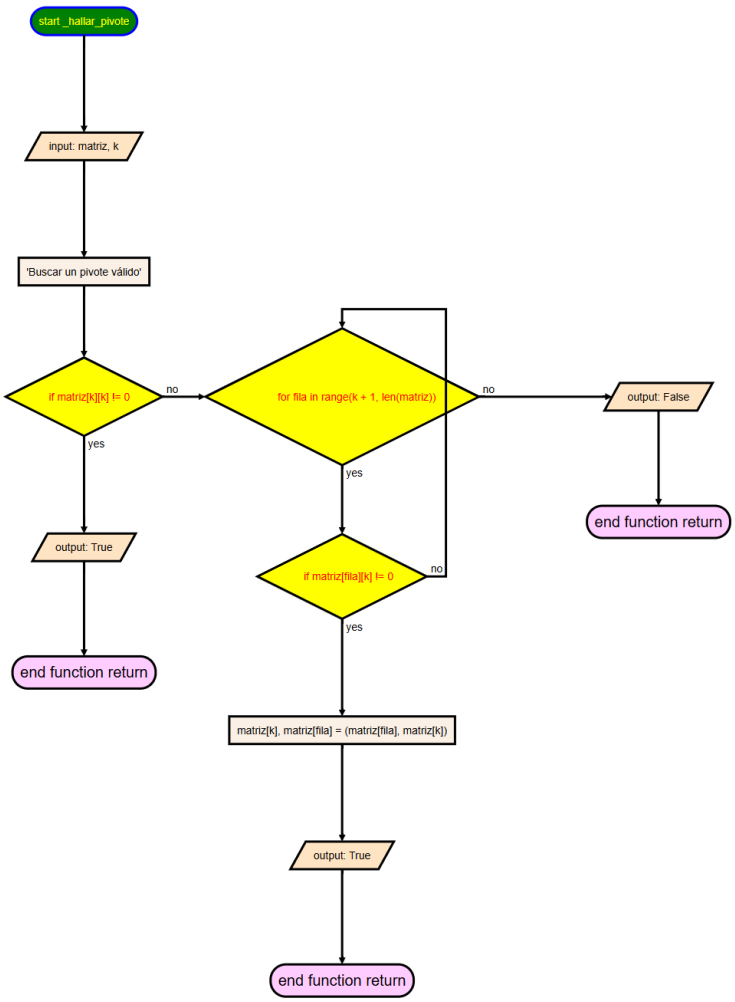
Leer Numero de Trapecios



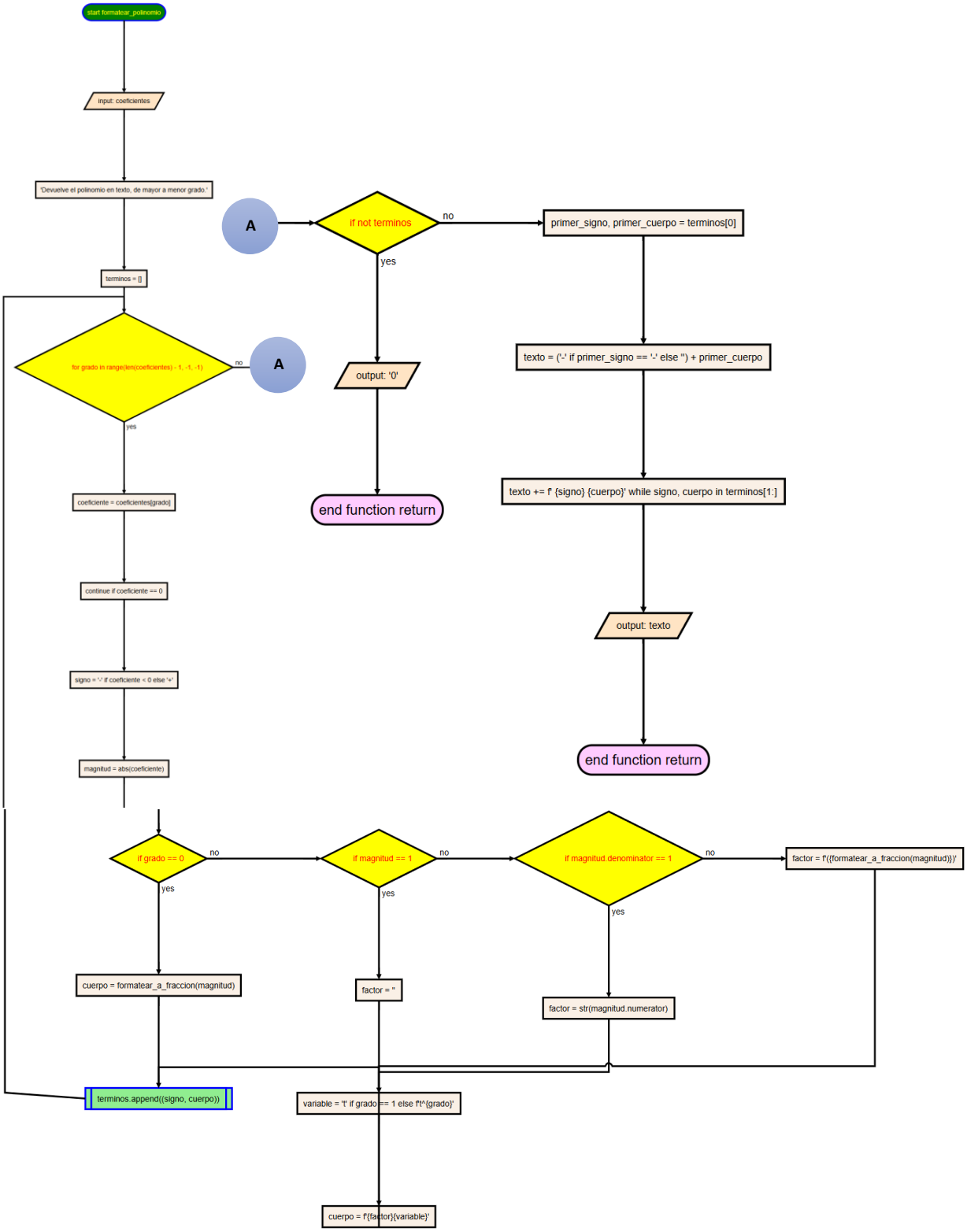
Leer Fracción



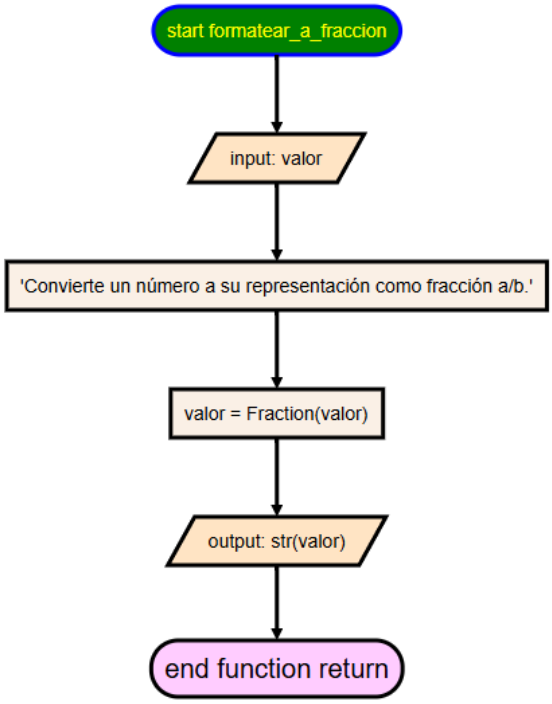
Hallar Pivote



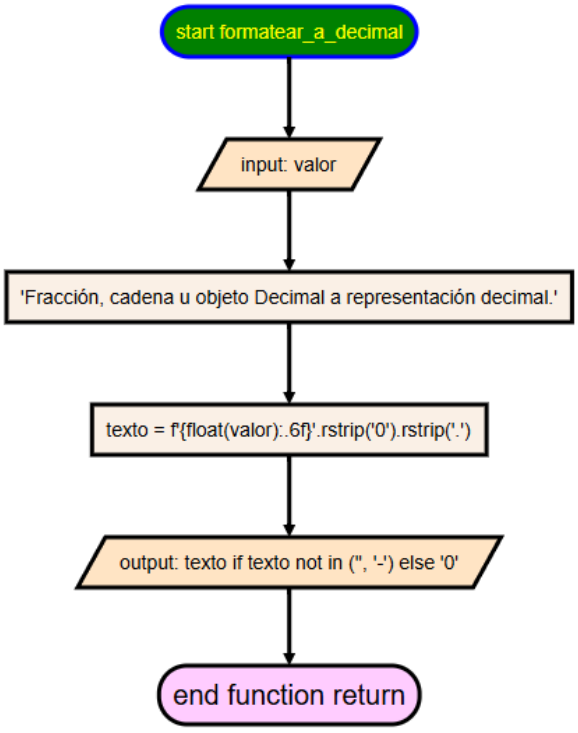
Formatear Polinomio



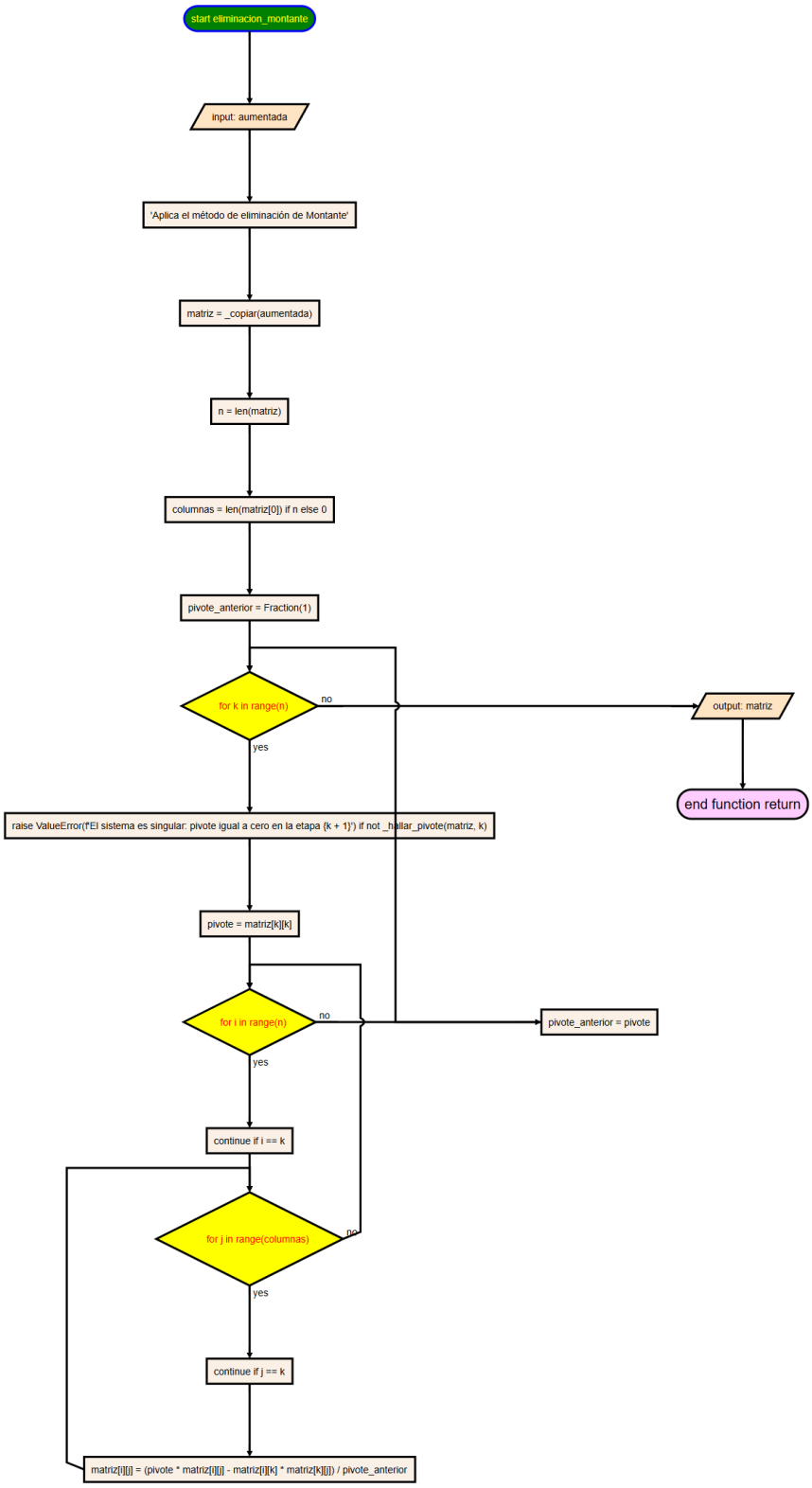
Formatear Fracción



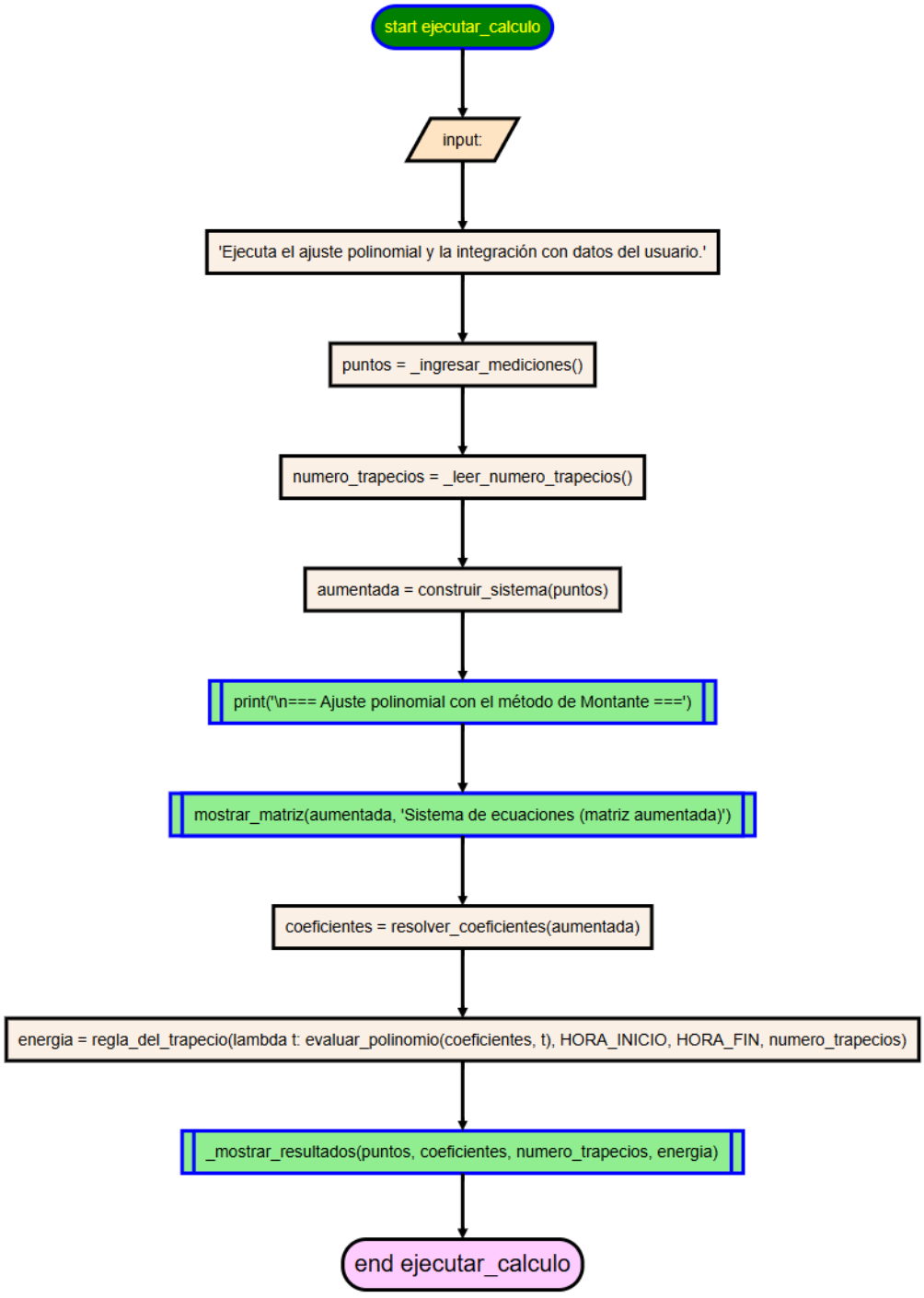
Formatear Decimal



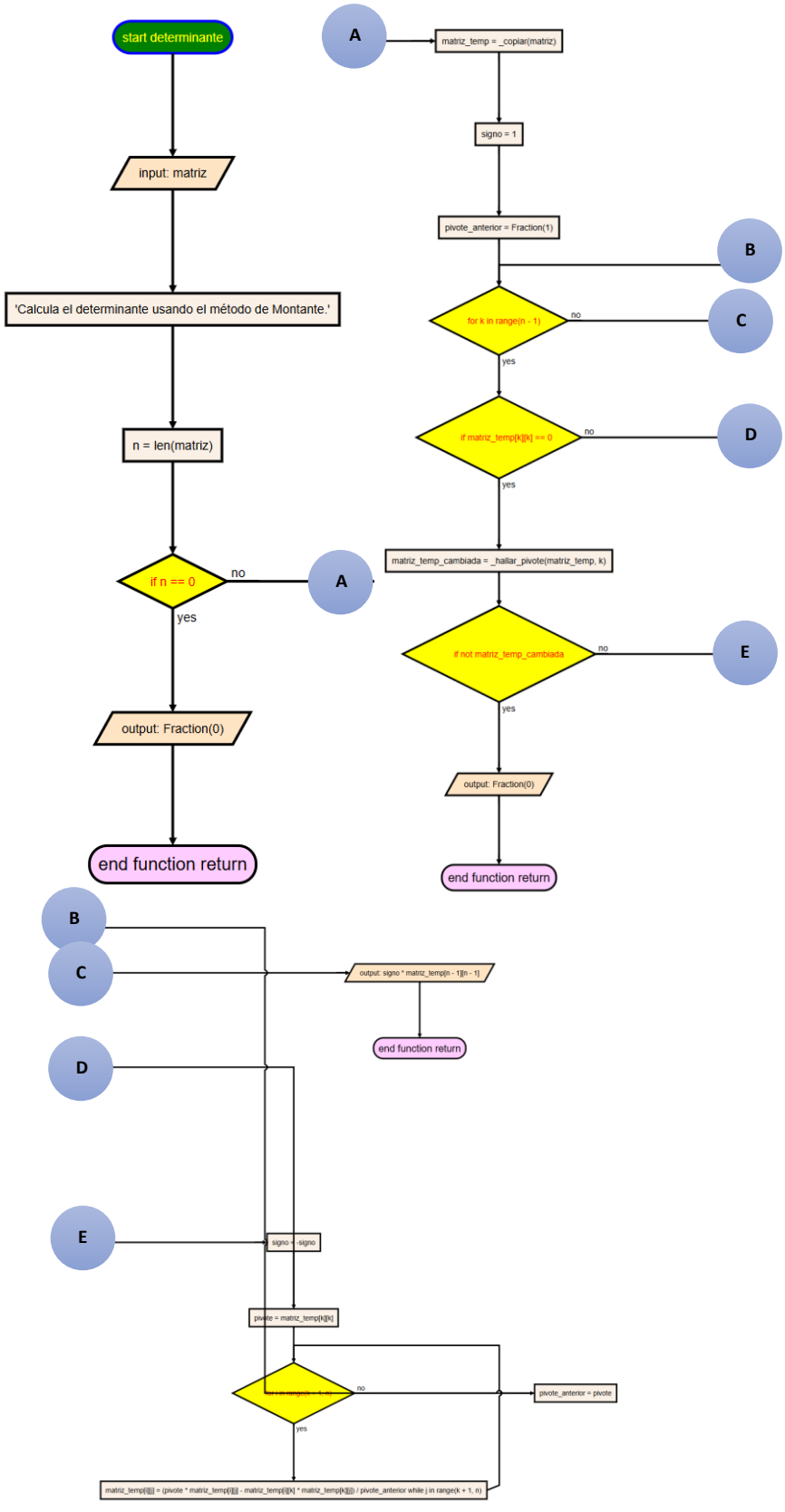
Eliminación Montante



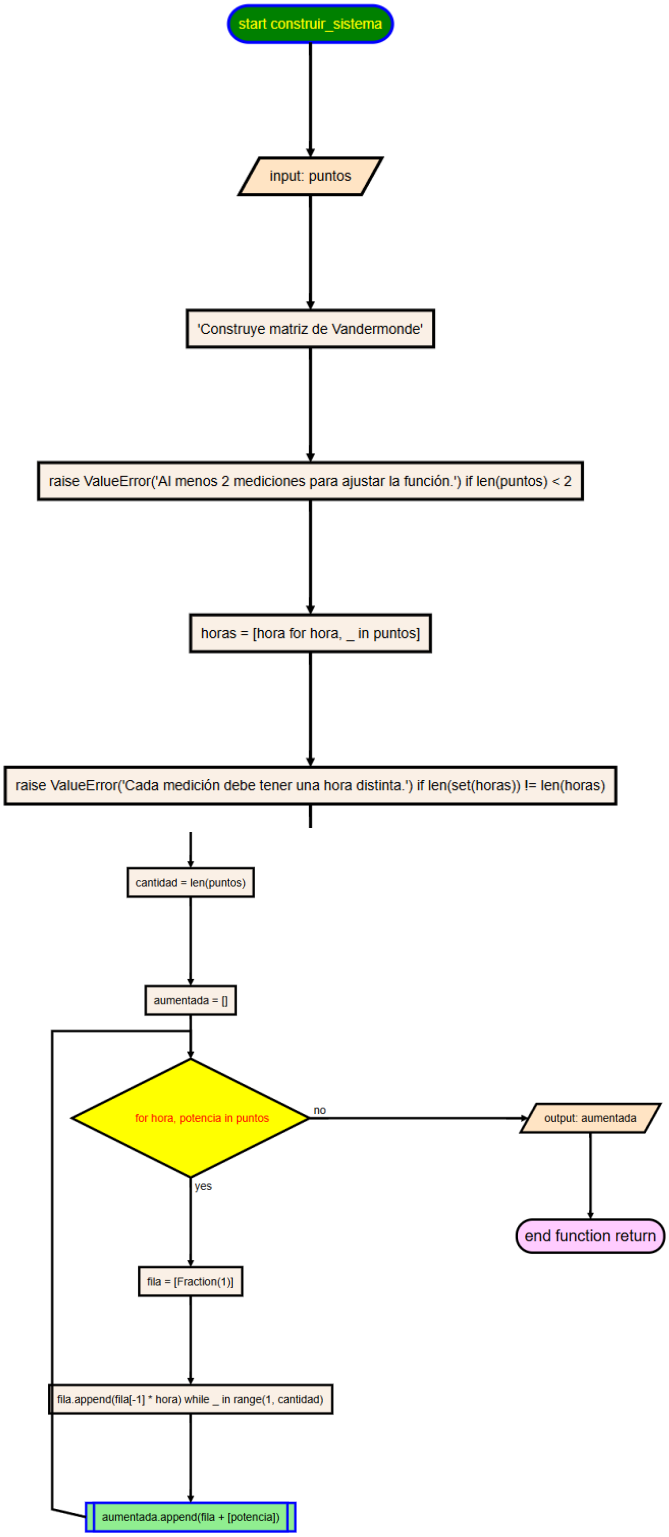
Ejecutar Cálculo



Determinante



Construcción del Sistema



Conclusión general

Con el desarrollo de este proyecto se aplicaron métodos numéricos para resolver un problema relacionado con el funcionamiento de las baterías de respaldo de un centro de datos. A partir de mediciones de potencia en diferentes horas, se construyó un polinomio de interpolación mediante el sistema de Vandermonde y se resolvieron sus coeficientes utilizando el método de Montante.

Posteriormente, se utilizó la regla compuesta del trapecio para aproximar la integral de la función de potencia durante un periodo de 24 horas. Esta integral permite estimar el cambio neto de energía: un resultado positivo indica que predominó la carga de las baterías, mientras que uno negativo indica que predominó la descarga.

Además de aplicar los procedimientos matemáticos, el proyecto permitió organizar el programa en módulos, validar los datos ingresados y presentar los resultados de manera comprensible. Se concluye que los métodos implementados son útiles para obtener una estimación del balance energético a partir de mediciones discretas. Sin embargo, el resultado depende de los datos proporcionados y del polinomio interpolador; cuando las mediciones no cubren todo el día, el modelo se extrapola a las horas restantes, por lo que la estimación debe interpretarse considerando esa limitación.

Ejemplo de ejecución

